Laboration 4 – Hashtabell (Öppen adressering)

Viktigt! *Som vanligt, läs "Information och regler kring laborationerna".*

Din lösning ska lämnas in i Canvas, gör inlämningn så snart du känner dig färdig med den. Du ska också lämna in en screenshot där det framgår att testfunktionen har körts utan att några asserts aktiverats samt en screenshot/fil där det framgår att du gjort deluppgift 4.3.

Läs varje deluppgift noggrant då det är viktigt att du följer instruktionerna för att du ska få ihop allt rätt. Om du undrar över något eller tycker att något är otydligt, hör av dig och fråga.

Vid inmatning från användaren kan du anta att korrekt typ matas in.

Se till att vara förberedd inför labbtillfällena, så att du verkligen hinner få hjälp med det du eventuellt har problem med.

Tänk igen om hur du ska ta dig an uppgiften och planera lösningen innan du börjar koda – vad är funktionens uppgift och vad måste hända rent logiskt. Rita gärna upp olika scenarion för att förstå vilka problem som kan uppstå.

Uppgift 4.1: Ladda ner och kompilera labbskelettet

Hämta filerna i den zippade filen Hashtable.zip som finns i Modul E i Canvas. Öppna sedan ett nytt projekt i din utvecklingsmiljö. Lägg till filerna i projektet. Programmet ska kompilera korrekt (med eventuella varningar). Det kommer dock inte att köra köras korrekt.

Obs! De två testfilerna ovan är samma fil, den som har tillägget "updated" är uppdaterad med några mindre detaljer (alla collision-variabler är unsigned, det finns ett case 8 i menyn, case 6 i menyn använder %u för att skriva ut unsigned int istället för %d, vid deklaratiionen av htable i menyn initieras dess table-fält till NULL).

Uppgift 4.2: Implementera och testa din Hashtabell

Hashtabellen du ska implementera representeras av en dynamisk array av Buckets. En Bucket är ett nyckel-värde-par. I detta fall kommer nyckeltypen vara ett personnummer (int) och värdetypen Person. Typen Bucket hittar du i `Buckets.h` och får inte ändras och typen Person hittar du i `Person.h`. Hashtabellen kommer alltså att länka personnummer till en person.

Din uppgift är att implementera funktionerna i interfacet till HashTable som du hittar i filen `HashTable.h`. Tänk på att arrayen som representerar Hashtabellen ska allokeras dynamiskt i `createHashTable()`.

I uppgift 4.3 nedan ska du jämföra olika storlekar på hashtabellen med avseende på antalet kollisioner vid insättning. Det är därför viktigt att funktionerna `linearProbe()` och `insertElement()` returnerar antalet kollisioner enligt kommentarerna i kodskelettet. Räkna samtliga krockar, om hashindexet är upptagen och vi måste söka 4 steg framåt för att hitta en ledig plats så är det alltså totalt 4 krockar. Varje krock ökar risken för att det ska uppstå fler krockar.

<u>Exempel på att räkna krockar</u>
Den bucket som ska läggas till ska läggas på index x, den platsen är upptagen == 1 krock (totalt 1 krock)
Leta framåt ett steg, på index x+1 är det också upptaget == 1 krock till (totalt 2 krockar)
Leta framåt ett steg, på index x+2 är det också upptaget == 1 krock till (totalt 3 krockar)
Leta framåt ett steg, på index x+3 är det också upptaget == 1 krock till (totalt 4 krockar)
Leta framåt ett steg, på index x+4 är det ledigt och där läggs bucketen till --> det har skett totalt 4 krockar för insättningen.

I `HashTable.c` finns också en tom hash-funktion som du ska implementera.

I filen `test_HashTable.c` finns en meny-funktion du kan använda för att testa din hashtabell allteftersom du implementerar dess funktioner. I meny-funktionen kan du välja att mata in persondata manuellt eller att slumpvis generera datat.

När du implementerat alla funktioner i din hashtabell ska du köra funktionen `test()` som finns i filen `test_HashTable.c` som testar all funktionalitet. Om testfunktionen körs felfritt så fungerar alltså din hashtabell som den ska.

Hashtabellen ska lösa krockar med hjälp av öppen adressering (linjär sondering / linear probing).

Uppgift 4.3: Analysera hashtabellen

När du nu har en fungerande implementation av en hashtabell (som löser krockar med öppen adressering) ska du jämföra olika storlekar på hashtabellen med avseende på antalet krockar.

Kör funktionen `compareHashTableSizes()`, du blir då ombedd att ange hur många tabeller du vill jämföra samt att ange storlek för samtliga av dessa tabeller. Samma datamängd (50 slumpade personer) kommer att läggas till i hashtabeller av samtliga valda storlekar och antalet krockar räknas. Detta görs 1000ggr (med olika, slumpade, datamängder varje gång), antalet krockar summeras och ett medelvärde för varje tabellstorlek presenteras på skärmen.

Om du vill kan du ändra makrona SIZE och LOOPS. SIZE anger antalet personer som läggs till i hashtabellerna och LOOPS anger hur många körningar som görs på valda storlekar för tabellerna. Du kan t.ex. öka SIZE för att jämföra tabellstorlekarna på större datamängder än 50 och LOOPS för att köra ett annat antal varv än 1000.

Jämför några olika storlekar och försök hitta en som ger så få kollisioner som möjligt utan att slösa på allt för mycket minne.

Fundera på varför antalet krockar för olika storlekar på hashtabellen blir som de blir.